

The inherent adversarial robustness of analog in-memory computing

Received: 10 May 2024

Accepted: 23 January 2025

Published online: 19 February 2025

 Check for updates

Corey Lammie¹ , Julian Büchel¹ , Athanasios Vasilopoulos¹ ,
Manuel Le Gallo¹  & Abu Sebastian¹ 

A key challenge for deep neural network algorithms is their vulnerability to adversarial attacks. Inherently non-deterministic compute substrates, such as those based on analog in-memory computing, have been speculated to provide significant adversarial robustness when performing deep neural network inference. In this paper, we experimentally validate this conjecture for the first time on an analog in-memory computing chip based on phase change memory devices. We demonstrate higher adversarial robustness against different types of adversarial attacks when implementing an image classification network. Additional robustness is also observed when performing hardware-in-the-loop attacks, for which the attacker is assumed to have full access to the hardware. A careful study of the various noise sources indicate that a combination of stochastic noise sources (both recurrent and non-recurrent) are responsible for the adversarial robustness and that their type and magnitude disproportionately effects this property. Finally, it is demonstrated, via simulations, that when a much larger transformer network is used to implement a natural language processing task, additional robustness is still observed.

Deep neural networks (DNNs) have revolutionized machine learning (ML) and formed the foundation for much of modern artificial intelligence (AI) systems. However, they are susceptible to a number of different types of adversarial attacks, which aim to deceive them by giving them false information. These include poisoning¹, extraction², and evasion-based attacks³. Evasion-based attacks target DNN models by generating adversarial inputs, which achieve a desired (usually malicious) outcome when inferred². For example, carefully crafted adversarial inputs can consistently trigger an erroneous classification output from a DNN model. Hence, there is significant interest in developing algorithms and underlying compute substrates that make DNNs more robust to adversarial attacks^{3–5}.

One such compute substrate which shows significant promise for adversarial robustness is that based on analog in-memory computing (AIMC)^{6–8}. These architectures were primarily motivated by the need to minimize the amount of data transferred between memory and processing units during the execution of multiply and accumulate (MAC)-dominated workloads, to avoid the well-known memory-wall problem^{9,10}. DNN inference workloads are mostly comprised of MAC

operations. Consequently, when AIMC-based hardware is used for their acceleration, their energy-efficiency and latency can be greatly improved. For example, in an AIMC-based accelerator based on nanoscale non-volatile memory (NVM) devices, synaptic unit cells comprising one or more devices can be used to encode weights of DNN layers, and the associated matrix-vector multiplications (MVMs) operations can be performed in-place by exploiting Kirchhoff's circuit laws. When multiple crossbar arrays with physically stationary synaptic weights and digital computing units are connected using a fabric able to route data, it is possible to realize complete (end-to-end) inference workloads. A key drawback of these accelerators is that they typically exhibit a notable degradation in accuracy compared to their full precision counterparts, on account of device-level noise and circuit non-idealities. However, Hardware-Aware (HWA) training, where the DNN is made robust via the injection of weight noise during the training process, has been found to recover much of the accuracy loss^{11,12}.

One often overlooked benefit of the stochasticity associated with AIMC is robustness against adversarial attacks. In fact, prior work has demonstrated that even the robustification of DNN models introduced

IBM Research Europe, Rüschlikon, Switzerland. ✉ e-mail: corey.lammie@ibm.com; ase@zurich.ibm.com

by HWA training could provide a certain level of adversarial robustness^{13–16}. It has also been demonstrated that simulated hardware non-idealities yield adversarial robustness to mapped DNNs without any additional optimization¹³ and simulated adversarial attacks crafted without the knowledge of the hardware implementation are less effective in both black- and white-box attack scenarios^{14,15}. Novel DNN training schemes have been presented¹⁶, which improve robustness when evaluated using a number of simulated DNNs with measured noise data obtained from three different resistive random-access memory (RRAM)-based in-memory computing (IMC) prototype chips. Additionally, simulations have been used to investigate the relationships between adversarial robustness, HWA training, non-idealities in crossbars, analog implementation of activation functions, and supporting circuits¹⁷. Finally, attacks have been proposed to generate adversarial examples leveraging the non-ideal behavior of RRAM-based devices^{18–21}. However, none of these studies have been verified on real AIMC-based hardware. Moreover, it is not well understood, to what degree, the different noise sources contribute to adversarial robustness.

In this paper, we investigate the inherent adversarial robustness, i.e., how inherently vulnerable a system is to various types of attack vectors, of AIMC for (i) a ResNet-based convolutional neural network (CNN) trained for image classification (CIFAR-10), and (ii) the pre-trained RoBERTa²² transformer network fine-tuned for a pair-based sentence textual entailment natural language processing (NLP) task (MNLI GLUE)²³.

Adversarial ML is a broad research field²⁴. While we confine the scope of this paper to the investigation of the inherent adversarial robustness of AIMC-based hardware against evasion-based adversarial attacks, there are many other different types of attack strategies which exist. Instead of generating adversarial inputs, poisoning- and back-door attacks corrupt models by modifying the data used to train them^{25,26}. Side-channel²⁷ and other types of attacks which take advantage of physical characteristics of systems²⁸ can be used to infer hidden information about a model and introduce malicious instability.

Simulation (for both networks) and real hardware experiments (for the ResNet-based CNN) are performed (see “Methods” section) using an AIMC chip based on phase change memory (PCM) devices. First, for the ResNet-based CNN, in agreement with prior findings, we observe that injecting noise during HWA training improves robustness to adversarial attacks. We further observe that when HWA-trained networks are deployed on-chip, this robustness is notably improved. It is investigated what characteristics of different noise sources contribute to this robustness, and demonstrated that the type and magnitude properties of stochastic noise sources are much more critical than their recurrence property. We consider the type as a binary attribute indicating whether, for a given noise source, its magnitude is determined as a function of the input. Additionally, we investigate the efficacy of hardware-in-the-loop attacks, and demonstrate that even these kinds of attacks are less effective when stochastic systems are targeted. Finally, using the RoBERTa transformer network, it is demonstrated through simulation that for this much larger network and different input modality, additional adversarial robustness is still observed.

Results

Experimental validation of adversarial robustness

To experimentally study the adversarial robustness, we employed a PCM-based AIMC chip with tiles comprising 256×256 synaptic unit cells²⁹. Each unit cell contains four PCM devices (see Fig. 1b). The weights obtained via HWA training are stored in terms of the analog conductance values of the PCM devices, where two devices are used to store the positive and negative weight components, respectively. The conductance variations associated with these conductance values are thought to be the primary reason for potential adversarial robustness. As shown in Fig. 1a, the intrinsic stochasticity associated with the

conductance variations is likely to make the design of an adversarial attack rather difficult. The conductance variations themselves fall into two different categories. There is a non-recurrent category that results from the inaccuracies associated with programming a certain analog conductance value. This is typically referred to as programming noise²⁹. Besides this non-recurrent noise component, there is a recurrent variation in the conductance values arising from $1/f$ noise³⁰ and random telegraph noise (RTN) characteristic of PCM devices. In subsequent sections, we will investigate the role of recurrent and non-recurrent noise sources with respect to adversarial robustness.

For this study, we consider three different types of adversarial attacks and five different target platforms. The different types of attacks are (i) Projected Gradient Descent (PGD)³¹, (ii) Square³², and (iii) OnePixel³³. The attacks range from targeting small (localized) to larger (for some attacks, entire) input regions. Specifically, the PGD attack was chosen, as it is equivalent to the Fast Gradient Sign Method (FGSM) attack when the starting point is not randomly chosen and the L_∞ norm is used³⁴. Hence, it is superior to FGSM when more than one iteration is used to generate adversarial inputs. The Square and OnePixel attacks do not rely on local gradient information, and thus, are not affected by gradient masking, which is a defensive strategy aimed at diminishing the efficacy of gradient-based attacks by obfuscating the model’s loss function, rendering it less informative or harder to optimize³⁵. The OnePixel attack only permutes a small region (i.e., a single pixel) of the input, rather than a larger region, as typically targeted by other attacks.

The five different attack target platforms considered are: (i) the original floating-point model (Original Network (FP32)), (ii) the floating-point HWA retrained model (HWA Retrained (FP32)), (iii) a digital hardware accelerator with 4-bit fixed-point weight precision and 8-bit activation precision (Digital), (iv) a PCM-based AIMC chip model (AIMC Chip Model), and (v) a PCM-based AIMC chip (AIMC Chip). The PCM-based AIMC chip model is described in the “Methods” section and a match between the model and experimental data is provided in Supplementary Note 1. For the first target, an existing pre-trained model is used. For the second target, the pre-trained model is retrained using HWA training. For the third, fourth, and fifth targets, the parameters of the retrained model are mapped to the corresponding hardware and deployed. All auxiliary operations are performed in floating-point precision.

To determine the effectiveness of the adversarial attacks, different evaluation metrics can be used. Typically, the clean (baseline) accuracy is directly compared to the accuracy when adversarial inputs are inferred. In this paper, we adopt the Adversarial Success Rate (ASR) metric³⁶, which considers only samples that are classified correctly by the network. More specifically, the ASR is determined as follows. First, model predictions are determined for both clean and adversarial inputs. Using target labels of the clean inputs, the incorrectly classified clean inputs are identified. These target labels and the predictions of adversarial inputs (adversarial predictions), which originated from the incorrectly classified clean inputs, are then masked and discarded. Traditional ML evaluation metrics can then be computed using the masked target labels and adversarial predictions. We concretely describe the determination of the ASR in terms of accuracy using Algorithm 1 (see “Methods” section).

By adopting the ASR metric, we can evaluate and compare the performance of adversarial attacks which are generated and/or evaluated using different target platforms. To demonstrate this ability, in Supplementary Table II, we compute both the test set accuracy and ASR during training for a floating-point ResNet-based CNN trained for CIFAR-10 image classification using two different types of attacks, PGD and OnePixel, with different parameters. These attacks are considered in all fore-coming experiments, and are described in greater detail below. We also investigate the adversarial robustness by simulating a larger image classification task with 10,000 output classes (iNaturalist 2021) using the WRN-50-2-bottleneck64 CNN and observe similar

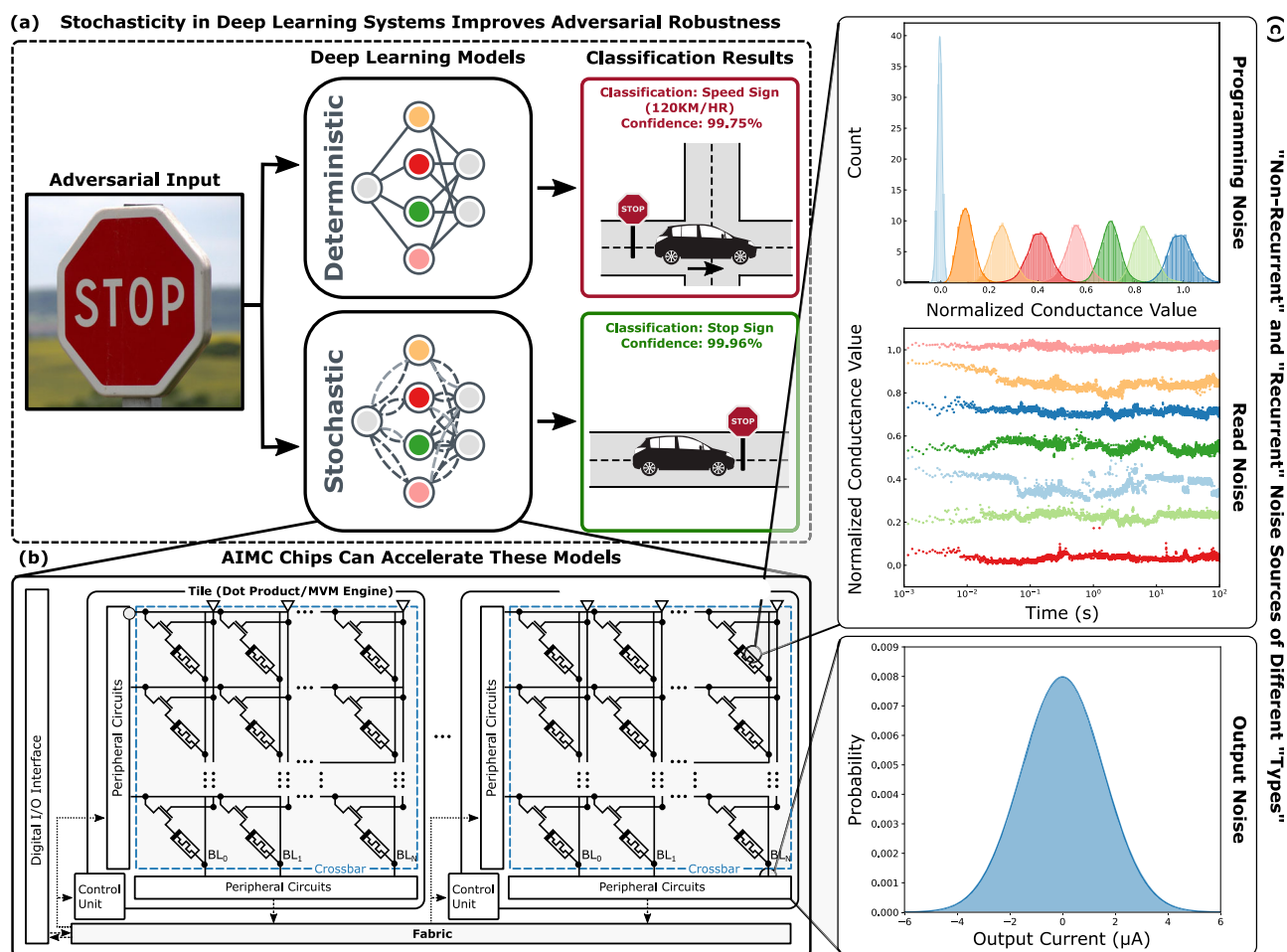


Fig. 1 | Adversarial robustness of analog in-memory computing. **a** An augmented (adversarial) image of a stop sign, intercepted and replaced by a malicious attacker, intended to be miscategorized as a speed sign, is fed into a (i) deterministic and (ii) stochastic DNN accelerator residing in an autonomous vehicle. The deterministic system incorrectly identifies the input as a speed sign, whereas the stochastic system correctly identifies the input as a stop sign. **b** Analog in-memory computing chips which can be used to execute DNN inference workloads are inherently stochastic. The circuits and devices they are constructed of introduce many different linear and non-linear noise sources, including, but not limited to: quantization noise, weight programming noise, weight read noise, circuit noise,

and temporal weight drift. **c** Noise sources have a number of distinct properties. The recurrence property determines whether a noise source is “non-recurrent” or “recurrent.” Non-recurrent noise sources introduce noise once (usually when the system is configured), while recurrent noise sources introduce noise at different frequencies during normal system operation. The type property determines whether the noise magnitude is determined as a function of the input. As depicted, programming noise is non-recurrent and its magnitude is input dependent. Read noise is recurrent and its magnitude is input dependent. We model output noise such that it is recurrent and its magnitude is input-independent.

findings (see Supplementary Note 3). For the ASR to be truly agnostic to the target platform, it should be independent of the network accuracy. It is observed that the ASR is *sufficiently* independent of the test set accuracy and is only notably perturbed for extreme scenarios (i.e., when the test set accuracy is <25%).

For each type of attack, we vary two parameters. The first parameter relates to the number of attack iterations, whereas the second parameter relates to the attack magnitude, i.e., the maximum amount of distortion the attack can introduce to the input. When evaluating the ASR as a function of different attack parameters, especially for those relating to magnitude, caution must be taken so that the ground truth label is not changed by extreme permutations. If the severity of an attack or the number of attack iterations is too high, then the ground truth labels of the generated adversarial inputs can differ from the original inputs, which is problematic as the underlying semantics of the original input have now changed. Consequently, for all experiments, the maximum value for each parameter is determined by manually inspecting adversarial examples and determining the points at which the ground truth label is changed (see “Methods” section).

To comprehensively compare the adversarial robustness for all target platforms, while considering the number of attack iterations and magnitude, a contour or 3D plot, per target platform, is required. These are difficult to compare relative to each other. Hence, instead, we generate an ASR for each target platform, which is indicative of the average behavior of the objective as a function of both parameters. This is generated by projecting a straight line from the bottom-left to top-right corners of each contour plot. Along this line, the ASR is extracted. Each point in this line is associated with two values: one governing the number of attack iterations and another governing the attack magnitude. A dual x-axis is used to associate these.

For each evaluation platform, aside from the AIMC chip, the strongest attack scenario is considered, i.e., attacks are both generated and evaluated using the same platform. To both generate and evaluate attacks on the AIMC chip, hardware-in-the-loop attacks would have to be performed—such attacks are non-trivial to perform on stochastic hardware, and hence, instead, are the sole focus of the “Hardware-in-the-loop adversarial attacks” section. Consequently, for the AIMC chip, adversarial inputs are generated using the AIMC chip model. For the Software

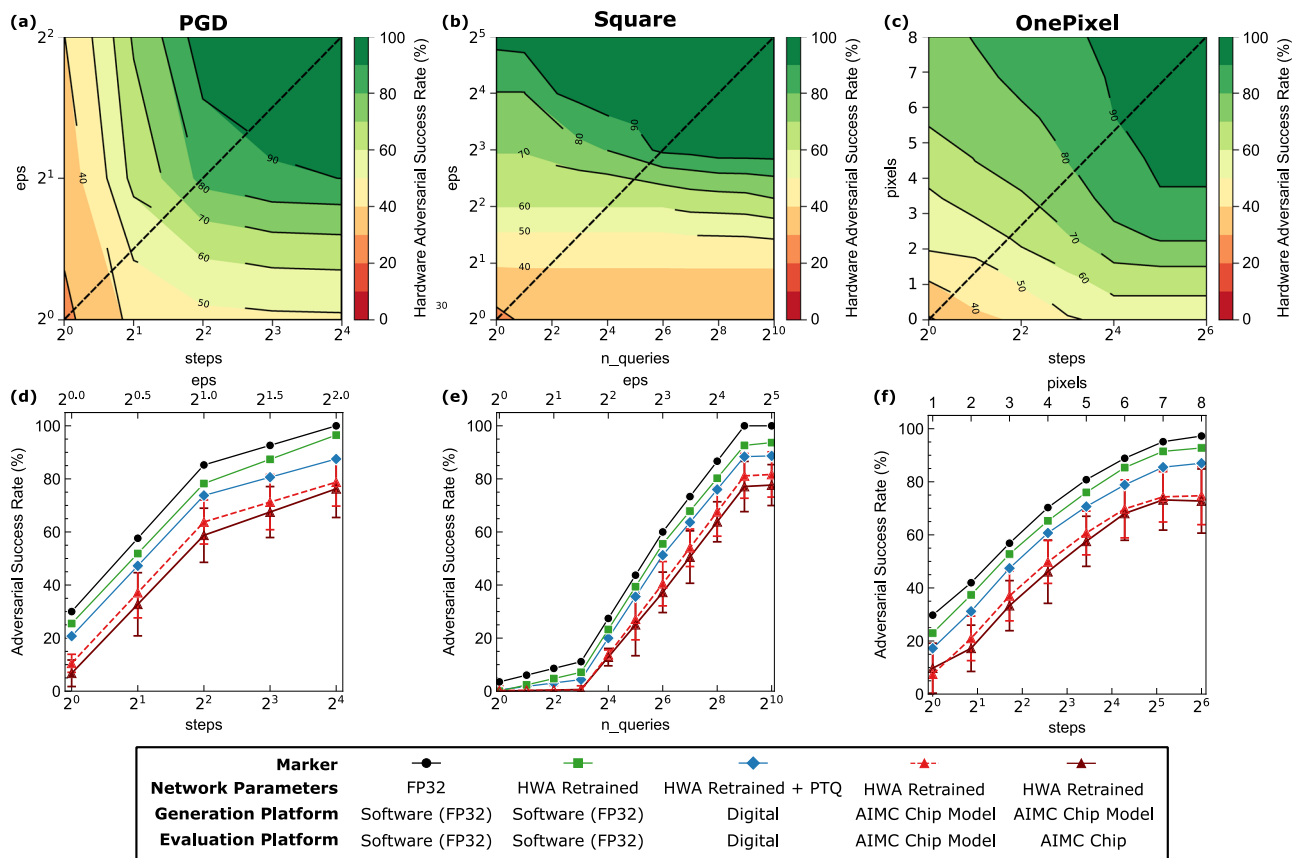


Fig. 2 | Experimental validation of adversarial robustness. **a–c** Adversarial inputs generated using the PGD, Square, and OnePixel adversarial attacks for different values of both attack parameters. The attacks are evaluated using the ASR metric for the AIMC chip evaluation platform. **d–f** For different configurations, denoted

using distinct marker and line styles, the ASR envelope (from the dashed lines in **(a–c)**) of each evaluation space is compared. For non-deterministic evaluation platforms, evaluation experiments are repeated $n = 10$ times. Mean and standard deviation values are reported.

(FP32) evaluation platform, attacks are generated using the same platform with FP32 and HWA Retrained (FP32) weights, respectively.

In Fig. 2a–c, the ASR quantity is related to both aforementioned parameters for the AIMC chip evaluation platform. As can be observed in Fig. 2d–f, the envelope for the floating-point HWA retrained model is smaller than the original floating-point model, meaning it is more robust. This result is consistent with prior work^{13,14,16,37}. The digital hardware accelerator has an even smaller envelope, meaning it is more robust than the floating-point HWA retrained model. It is noted that like the operation of the digital hardware accelerator, the evaluation of the floating-point HWA retrained model is deterministic. Contrastingly, the digital hardware platform is subject to quantization noise and accumulation error³⁸. While this error is introduced during normal system operation, as digital hardware is typically deterministic (as is assumed here), it can be predicted in advance, i.e., when adversarial inputs are generated. In fact, it has been demonstrated that adversarial attacks targeting these deterministic systems can be more effective³⁹. Critically, (i) there is a good agreement between the modeled ASR for the AIMC chip and the ASR rate of the hardware models on the AIMC chip, and (ii) the hardware experiments on the AIMC chip result in the smallest envelopes, and hence, the highest level of robustness to all investigated adversarial attacks.

Source of adversarial robustness

In this section, we present a detailed study of the various noise sources that contribute to the higher adversarial robustness observed experimentally on the AIMC chip (see “Methods” section). Specifically, we investigate the role of four different noise properties—the recurrence (or lack thereof), location, type, and magnitude. For this study, we rely

on the hardware model of the AIMC chip. As described earlier, non-recurrent noise sources introduce noise once during operation, i.e., when weights are programmed, whereas recurrent noise sources introduce noise multiple times during operation, i.e., when a MVM operation is performed. In the AIMC chip model, the recurrent noise sources are sampled multiple times (specifically per mini-batch), whereas non-recurrent noise sources are only sampled once. All noise sources are assumed to be Gaussian and centered around zero.

The non-recurrent component of stochastic weight noise primarily arises from programming noise. The recurrent components of the stochastic weight noise primarily arises from both device read noise ($1/f$ and RTN) and output noise, which is introduced at the crossbar output, before currents are read out using Analog-to-Digital Converters (ADCs). Output noise includes $1/f$ noise from amplifiers and other peripheral circuits, as well as other circuit non-linearities (IR drop, sneak paths, etc.) of small magnitude that can be approximated as a random perturbation on the crossbar output. Hence, in the model, non-recurrent weight noise is modeled, and at the output, these combined effects are modeled using additive recurrent noise of a fixed magnitude which is assumed to be independent of the total column conductance. For each configuration, other deterministic noise sources, e.g., input and output quantization, are also modeled. Other non-deterministic noise sources are assumed to contribute negligibly—as evidenced by the strong match between the AIMC chip and the AIMC chip model across all hardware experiments—and hence, are not modeled.

In order to assess how three properties of stochastic noise sources, namely, recurrence, location, and type affect adversarial robustness, we modify the behavior of the AIMC chip model to focus

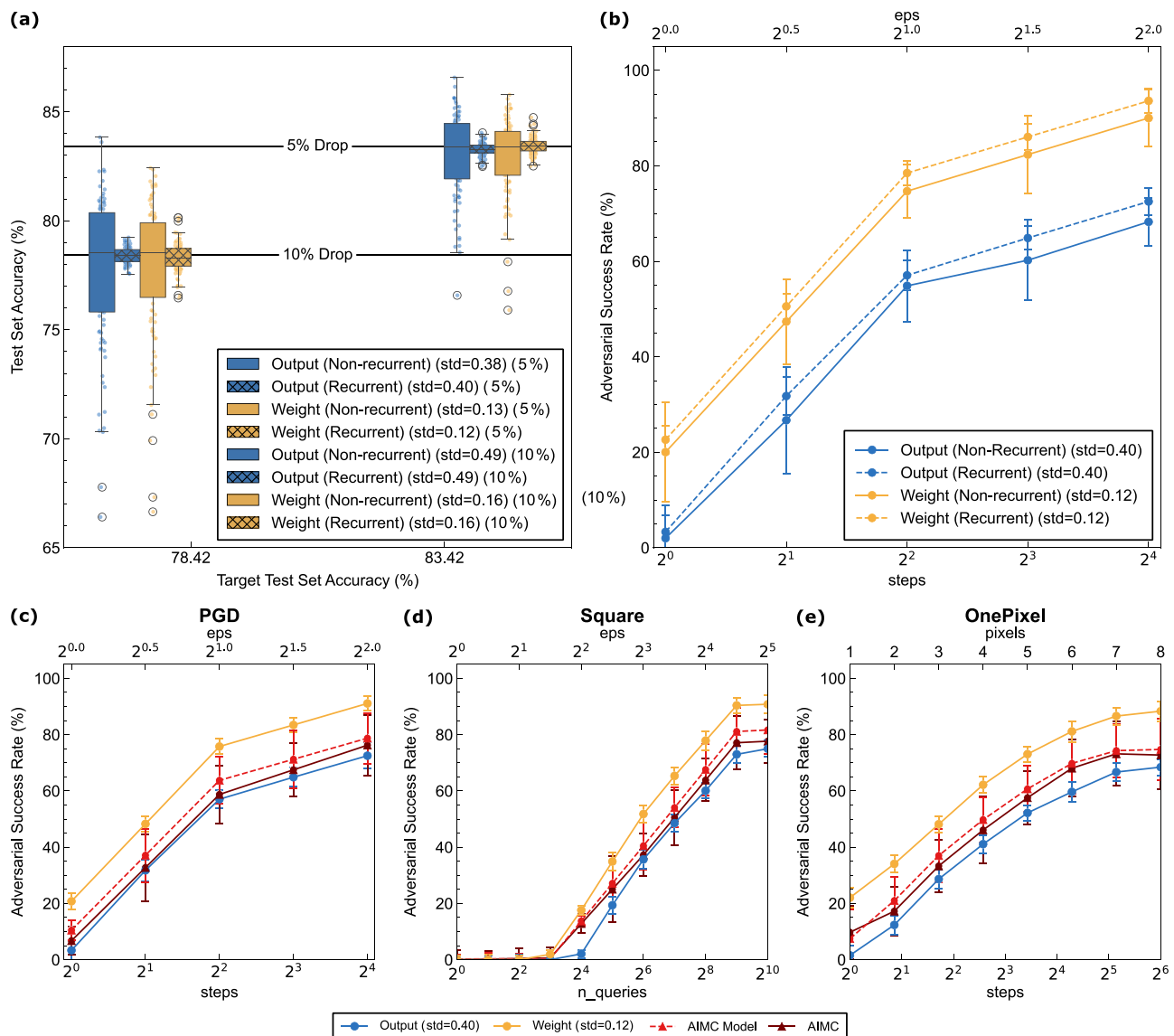


Fig. 3 | The source of adversarial robustness. **a** The test set accuracy for AIMC models where only output or weight noise is considered when the noise magnitude is set to result in test set accuracy drops (compared to the original floating-point model) of 5% and 10%. The test accuracy is evaluated for $n = 1000$ repetitions and mean and standard deviation values are reported. **b** For PGD, the robustness of both non-recurrent and recurrent output and weight noise that result in a 5% test

set accuracy drop is evaluated. **c–e** The ASR for the PGD, Square, and OnePixel attacks are compared for the AIMC models with only output and weight noise (both recurrent), the AIMC model, and the AIMC chip. For non-deterministic evaluation platforms, evaluation experiments are repeated $n = 10$ times. Mean and standard deviation values are reported.

exclusively on a single stochastic noise source. In Supplementary Note 4, we investigate how the inherent adversarial robustness of AIMC-based hardware is affected by temporal drift.

Separately, we investigate both recurrent and non-recurrent noise sources at two locations: the weights, whose effect is determined as a function of the input, and at the output, whose effect is input-independent. A total of four stochastic noise sources are considered: (i) recurrent output noise, (ii) recurrent weight noise, (iii) non-recurrent weight noise, and (iv) non-recurrent output noise. The effect of the noise magnitude is determined by, for each noise source, modifying the noise magnitude such that the resulting test set accuracy is lower than the floating-point test set accuracy by a desired percentage (i.e., drop). We consider drops of 5% and 10%.

In Fig. 3a, for a large number, $n = 1000$, of repetitions, the test set accuracy is reported for both non-recurrent and recurrent output and weight noise. In Supplementary Notes 2 and 5, we investigate adversarial robustness to varying degrees of stochasticity and combinations

of output and weight noise. It is observed that, for non-recurrent and recurrent noise sources normalized to produce the same test set accuracy, the noise magnitude is approximately equal, i.e., the recurrence property does not affect the resulting average test set accuracy. For non-recurrent noise sources, the variation of the resulting test set accuracy is larger. For noise sources normalized to produce a larger test set accuracy drop (10%), the noise magnitude is larger, compared to the smaller drop (5%). Intuitively, as the noise magnitude is increased, the test set accuracy decreases and the network becomes more robust to adversarial attacks.

To investigate the effect of the recurrence, location, and type properties on adversarial robustness, further experiments are performed in Fig. 3b. Comparisons are made using one of the selected attacks, PGD. For the noise magnitudes associated with the smaller test set accuracy drop (5%), the ASR envelope is determined for $n = 10$ repetitions, for the four aforementioned noise sources. Two key observations can be made: (i) output noise exhibits greater adversarial

robustness compared to weight noise, and (ii) in addition to not affecting the average test set accuracy, the recurrence property does not affect the ASR. We postulate that weight noise leads to less robustness when normalized to produce the same error, as the effect of weight noise is input dependent, whereas output noise is not.

In Fig. 3c–e, we further compare the ASR envelope for the PGD, Square, and OnePixel attacks. It is observed that, on average, the model with only output noise exhibits the highest adversarial robustness. The model with only weight noise is the least robust. The AIMC chip model and AIMC chip, with test set accuracy values of 84.85% and 84.31%, respectively, i.e., drops of 3.57% and 4.11%, exhibit greater adversarial robustness than the modified AIMC chip model with only weight noise, but less adversarial robustness than the modified AIMC chip model with only output noise. From this analysis, it can be concluded that the type and magnitude properties of stochastic noise sources have the greatest influence on adversarial robustness. The location and recurrence properties have negligible influence.

Hardware-in-the-loop adversarial attacks

Next, we determine the efficacy of hardware-in-the-loop attacks, i.e., where it is assumed that the attacker has full access to the AIMC chip. We compare the efficacy of one white- and one black-box attack (PGD and OnePixel). White-box attacks are especially difficult to perform for stochastic hardware, as the construction of representative hardware models (with minimal mismatch) is non-trivial, and in many cases, even unfeasible. While automated ML-based in-the-loop modeling⁴⁰ approaches can be utilized, they require a significant amount of data, which is instance-specific. Hence, they are not considered in this paper.

For hardware-in-the-loop attacks, when a white-box attack is deployed, to perform backwards propagation, for each layer, weights and cached inputs are required¹. Additionally, the output(s) of the network is(are) required. As ideal, i.e., floating-point precision, weights cannot be programmed, there is some deviation between the target and programmed conductances, so the target weights (if known) cannot simply be used by the attacker. Additionally, read noise introduces random fluctuations when MVM workloads are executed. Representative weights can be inferred by solving the following optimization problem, as described in Büchel et al.⁴¹

$$\hat{\mathbf{G}} = \arg\min_{\hat{\mathbf{G}}} \sum_{b=1}^B \|\hat{\mathbf{G}}\mathbf{x}_b - \tilde{\mathbf{y}}_b\|_2. \quad (1)$$

Inputs to each layer can be cached during normal operation by probing input traces to Digital-to-Analog Converters (DACs). To perform backwards propagation, this information can be used to construct a proxy network graph (see “Methods” section), for which gradients can be computed in floating-point precision using the chain-rule. For the sake of practicality, an ideal backwards pass is assumed, i.e., straight-through-estimators are used for regions which are non-differentiable, and all values are assumed to be in floating-point precision. It is noted that, as the next candidate adversarial input to the network is usually dependent on the result of backwards propagation of the previous candidate input, this process cannot normally be pipelined. Consequently, depending on the operation speed of the chip, attacks generated using AIMC chips can be susceptible to low-frequency noise and temporal variations, such as conductance drift⁴². To mitigate these effects, as reprogramming all devices after each adversarial example is presented is not desirable, $\hat{\mathbf{G}}$ can be re-inferred. It is noted that, for black-box attacks, these effects cannot be effectively mitigated—except in the scenario where the attacker is aware of exactly when the chip was programmed.

In Fig. 4, we compare the efficacy of hardware-in-the-loop attacks generated and evaluated using the AIMC chip to three different attack scenarios, where: (i) adversarial inputs are generated and evaluated using the digital hardware accelerator, (ii) adversarial inputs are

generated and evaluated using the AIMC chip model, and (iii) adversarial inputs are generated using the AIMC chip model and evaluated using the AIMC chip. When evaluated using the AIMC chip, adversarial attacks generated using hardware-in-the-loop attacks on the AIMC chip are more effective than on the AIMC chip model. Hardware-in-the-loop attacks generated and evaluated on the AIMC chip model are marginally less effective. Critically, for all scenarios involving either the AIMC chip or AIMC chip model, additional adversarial robustness is observed compared to hardware-in-the-loop attacks both generated and evaluated on the digital hardware platform.

We emphasize that to take the required circuit measurements to perform hardware-in-the-loop attacks, even for black-box attacks, where the output logits are needed, significant knowledge about the underlying hardware is required. In some instances, direct traces may be unavailable, meaning that a combination of other signals must be used as a proxy, reducing attack efficacy. In others, developing a realistic hardware model is too time-consuming and not practically feasible. Similarly to when a representative hardware model is attacked, unlike for deterministic systems, e.g., digital hardware accelerators, the generated adversarial inputs are specific to that particular instance of the hardware, and hence not useful for large-scale attacks targeting many instances at the same time.

Applicability to transformer-based models and natural language processing tasks

Finally, to determine whether additional adversarial robustness is still observed for much larger transformer models and different input modalities, we simulate a pre-trained floating-point RoBERTa model fine-tuned for one GLUE task, MNLI. The RoBERTa model has ~125M parameters and the MNLI task comprises 393K training and 20K test samples (pairs of sentences). When fine-tuning the model for the down-stream MNLI task, HWA training was performed (see “Methods” section). This model exceeds the weight capacity of the IBM HERMES Project Chip, so instead of performing hardware experiments, simulations were conducted using the AIMC chip model.

A number of different hardware attacks for NLP tasks with text-based input exist. These usually target either specific characters, words, or tokens⁴³. One major challenge when generating adversarial attacks for NLP tasks is semantic equivalence. For images, small permutations are usually not perceivable. However, for text, small permutations are much more notable (even at a character level), and are more likely to alter the semantics of the input. We consider the Gradient-based Distributional Attack (GBDA) attack⁴⁴, which instead of constructing a single adversarial example, as done by most types of attacks, searches for an adversarial distribution and considers semantic similarity using the BERTScore metric⁴⁵. This enforces some degree of semantic equivalence. In Supplementary Table III, we list a number of adversarial text-based inputs for different λ_{sim} and n_{iters} parameter values, in addition to their respective BERTScore values.

Similarly to as done in previous sections, we consider varying two attack parameters, which relate to the number of attack iterations and the magnitude, n_{iter} and λ_{sim} , respectively. The λ_{sim} parameter is inversely proportional to the attack magnitude. In Fig. 5, the ASR is reported for the first four target platforms (listed in section). Additional robustness is once again demonstrated for the AIMC chip model for a range of attack parameter values and BERTScore values. This is significant, as it indicates that additional adversarial robustness is still observed for (i) different input modalities, and (ii) much larger and more parameterized networks.

Discussion

Adversarial attacks and other types of cyber attacks pose a significant threat to DNN models. Hence, a number of dedicated defense mechanisms⁴⁶ and training methodologies⁴⁷ have been proposed. The most effective defense mechanism, which is adversarial training, is too

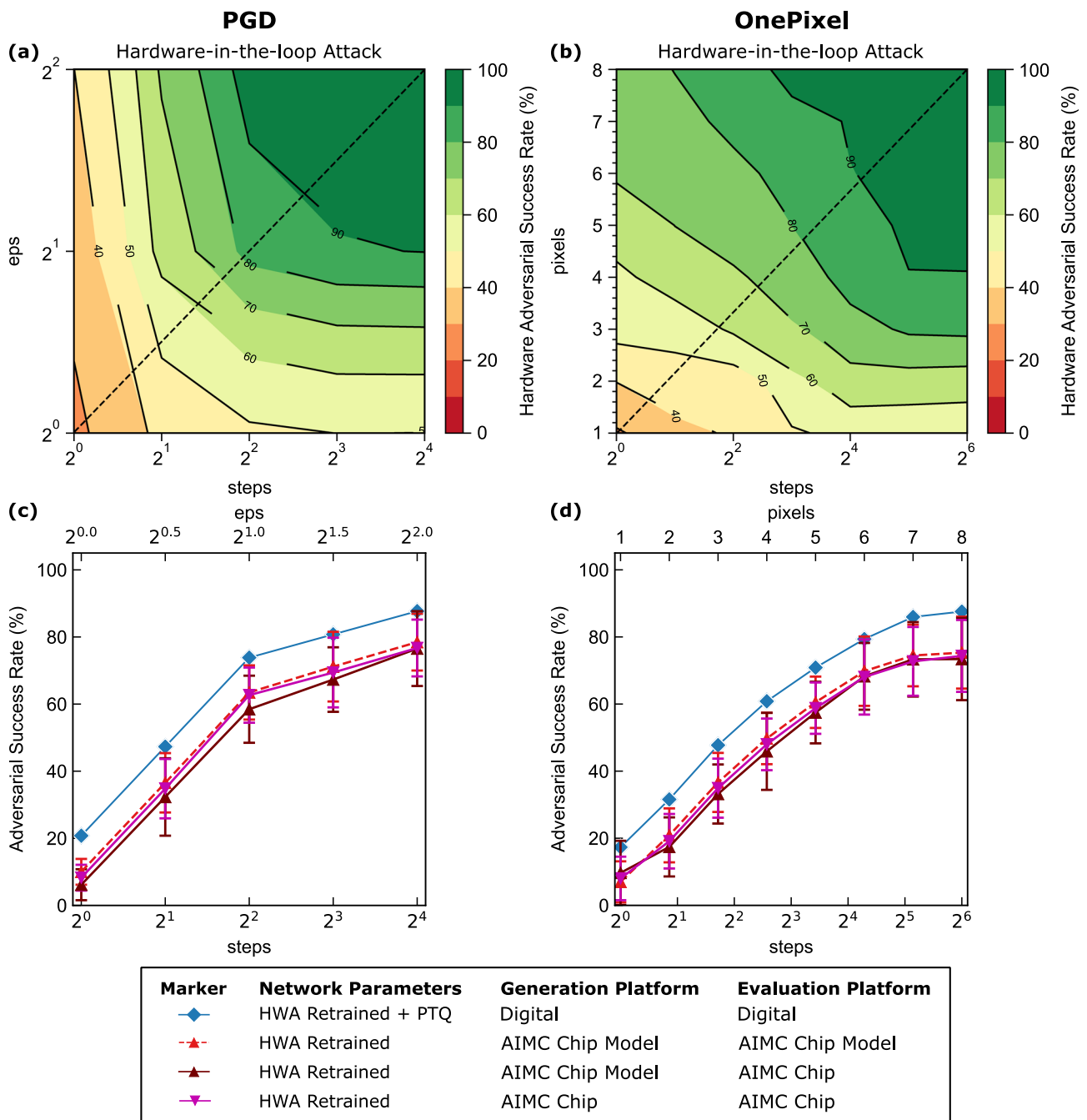


Fig. 4 | Robustness to hardware-in-the-loop adversarial attacks. **a, b** Adversarial inputs generated using the PGD and OnePixel adversarial attacks, both generated and evaluated using the AIMC chip. **c, d** When evaluated using the AIMC chip, adversarial inputs generated using hardware-in-the-loop attacks are more effective than when they are generated using the AIMC chip, compared to when they are generated using the AIMC chip model. Adversarial inputs generated and evaluated

using the AIMC chip model exhibit similar adversarial robustness to the AIMC chip. For all three configurations, additional adversarial robustness is observed compared to when they are generated and evaluated using the digital hardware accelerator. For non-deterministic evaluation platforms, evaluation experiments are repeated $n = 10$ times. Mean and standard deviation values are reported.

computationally expensive for practical deployment⁴⁸. Moreover, heuristic-based defenses have been demonstrated to be vulnerable to adaptive white-box adversaries. Therefore, their widespread adoption is limited and their use is not often considered. In both black- and white-box settings, when the attacker does not have unrestricted access to the hardware, we have demonstrated, empirically, both non-recurrent and recurrent stochastic noise sources influence robustness to adversarial attacks. These stochastic noise sources, which are present in AIMC chips, inherently act as an effective defense mechanism against adversarial attacks, even under different parameter

configurations (see Supplementary Note 6). When the attacker has unrestricted access to the hardware, we demonstrate that in addition to being much more difficult to effectively attack, inherent adversarial robustness is also observed.

For AIMC chips, this increased robustness comes at no additional hardware cost, and does not require any augmentation to the training and deployment pipelines typically employed for DNNs to be accelerated using this hardware. Moreover, the inherent stochasticity of these chips makes them difficult to target using hardware-in-the-loop-based attacks. While other dedicated defense mechanisms may be

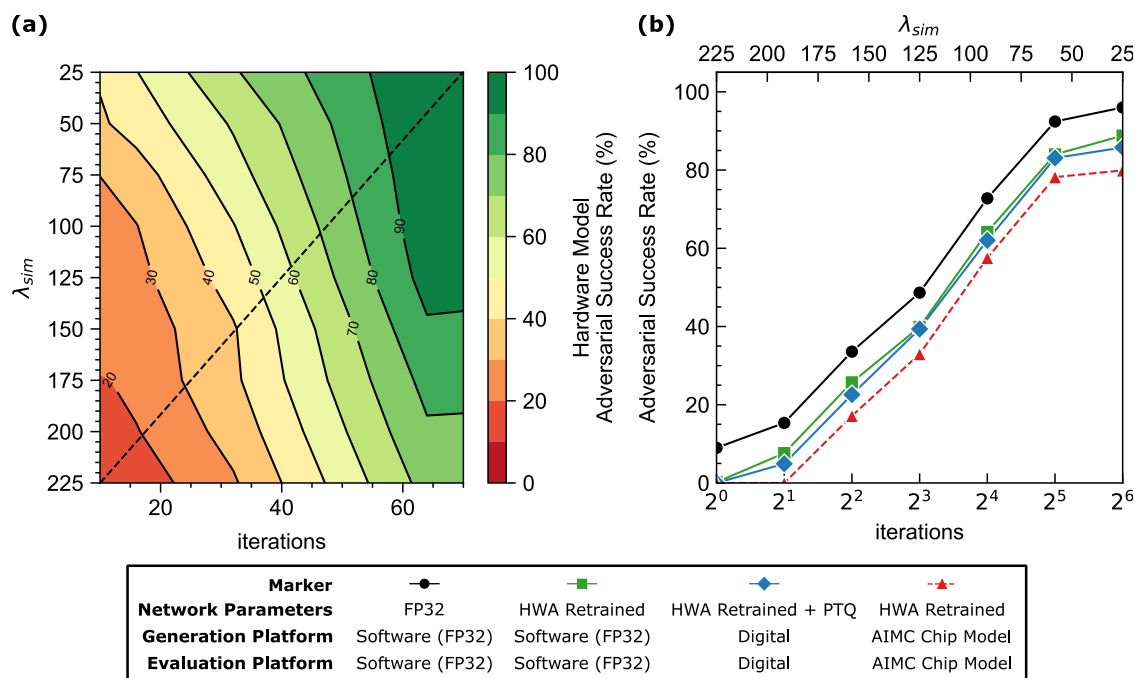


Fig. 5 | Robustness of transformer-based models and natural language processing tasks to adversarial attacks. a Using the GBDA adversarial attack, adversarial inputs are both generated and evaluated using the AIMC chip model. The ASR is reported for different values of both attack parameters (iterations and

λ_{sim}). **b** The ASR envelope is reported for a number of strongest attack configurations, which are denoted using distinct marker and line styles, the ASR envelope of each evaluation space is compared.

more effective, they incur some additional hardware/resource cost, and most critically, require modification to training and/or deployment pipelines. This adversarial robustness highlights yet another powerful computational benefit of the intrinsic stochasticity associated with AIMC chips similar to prior demonstrations such as in-memory factorization⁴⁹, Bayesian⁵⁰, and combinatorial optimization⁵¹.

Looking forward, as device and circuit technologies, and strategies for the mitigation of stochastic behavior continue to improve, it is expected that the inherent adversarial robustness of these chips will decrease. We demonstrate, that, particular types of stochastic noise sources, which introduce a relatively small additional error, at no additional resource cost, can be used to effectively defend against adversarial attacks. To improve adversarial robustness, for future AIMC designs, the presence, magnitude, and location of these noise sources, could be considered. For hardware accelerators based on other technologies, these types of noise sources could intentionally be introduced, at the cost of a small degradation in accuracy, in lieu, or in conjunction with, other adversarial defense mechanisms.

To conclude, in this paper, we performed hardware experiments using an AIMC chip based on PCM devices. Additionally, we performed simulations grounded by experimental data gathered from this chip to determine what characteristics of different noise sources contributed to this robustness, and evaluated the adversarial robustness of larger networks with different input modalities. To perform all these experiments, we developed a standardized and extendable methodology to evaluate the adversarial robustness of AIMC chips. With little effort, the ASR metric and our attack generation and evaluation methodology can be repurposed to evaluate the adversarial robustness against other attack types, other types of AIMC hardware, e.g., RRAM-based AIMC chips, and chips with a different underlying architecture. All types of NVM devices are susceptible to some degree of conductance drift, meaning that as long as NVM devices are used, interleaving samples during evaluation will counteract the effects of drift. The optimization problem formulated in Eq. (1) can also be repurposed for any arbitrary programmable conductive device,

meaning that our hardware-in-the-loop attack methodology can also be applied to other types of AIMC hardware, or with more effort, to other emerging technologies.

In addition to evaluating the adversarial robustness of different AIMC hardware configurations and technologies, future work entails the investigation of different attack types, e.g., poisoning-based attacks, and physical-based attacks, e.g., those which maliciously introduce instability to the system. While in this paper, due to the current nascent state of the field, the scope of available hardware data is relatively limited, in future, as it matures, data will become more readily available. Aside from evaluating the adversarial robustness of more attacks and larger networks using real hardware, future work can broaden this scope in several different ways. In addition to evasion-based attacks, adversarial training and poisoning attacks could be investigated—this is particularly due to the additional HWA retraining step that is required to achieve significant performance on AIMC hardware. A larger number of different configurations could also be investigated, in addition to the interaction of inherent adversarial robustness and stochastic dropouts^{52,53}. Finally, situations where the system is stable could be investigated, e.g., depending on the reliability of the system, the programming cycles used to program could be attacked.

Methods

Datasets and neural network models

Two datasets were used for evaluation: (i) CIFAR-10, an image classification task comprising 50,000 training images and 10,000 test images (32×32 pixel Red Green Blue (RGB)), and (ii) the NLI corpus, which is comprised of 443K sentence pairs annotated with textual entailment information. These are labeled as either neutral, contradiction, or entailment. For image classification, standard input pre-processing steps were performed to normalize inputs to zero-mean with a single standard deviation. The training set was used to construct a validation and training set for training the ResNet9 network, and the test set was used for evaluation. For sentence pair annotation,

standard pre-processing and tokenization steps were performed. The pre-trained RoBERTa transformer network, which was originally trained in floating-point precision using five English-language corpora of varying sizes and domains, totaling over 160GB of uncompressed text²², was fine-tuned using the matched MNLI validation set and evaluated using the mismatched MNLI validation set.

Neural network model training

First, both neural network models were trained in floating-point precision. The ResNet9S network was trained from randomly initialized weights using the CIFAR-10 training set. This was split into training and validation subsets with 40,000 and 10,000 samples each, respectively. A batch size of 128 was used with a cosine-annealing learning rate schedule. An initial learning rate of 0.1 and momentum value of 0.7 were used. The pre-trained-tuned RoBERTa model was fine-tuned using the matched MNLI validation set with a maximum sequence length of 256, a fixed learning rate of 2E-5, and a batch size of 16 for 10 epochs.

Next, for both the low-precision fixed-point digital and PCM-based IMC accelerators, HWA retraining was performed. The IBM AIHWKIT was used to inject noise during forward propagations. We refer the reader to Le Gallo et al.⁵⁴ for a comprehensive tutorial on HWA training using IBM AIHWKIT. The `InferenceRPUConfig()` preset was used with the `PCMLikeNoiseModel` phenomenological inference model. The following additional modifications were made to the simulation configuration: (i) biases were assumed to be digital (i.e., they are not encoded on the last column of AIMC tiles). (ii) Channel- (i.e., column-) wise weight scaling, which has a negligible performance impact, was performed. Weights were mapped to a per-channel conductance range during learning after each mini-batch. (iii) The size of each AIMC tile was assumed to be 256×256 . (iv) During training, multiplicative Gaussian noise was applied to unit weight values, with a standard deviation of 0.08, extracted from hardware measurements. For the AIMC chip and AIMC chip model, the HWA-trained weights were directly deployed. To deploy weights on the digital hardware model (described below), Post Training Quantization (PTQ) was performed on the HWA-trained weights using the default options as described at https://github.com/Xilinx/brevitas/tree/master/src/brevitas_examples/imagenet_classification/ptq using flexml.

Hardware models

Two hardware models were used, representative of (i) a low-precision fixed-point digital hardware accelerator with 8-bit activations and 4-bit weights, and (ii) the IBM HERMES Project Chip²⁹ (a PCM-based IMC accelerator).

Low-precision fixed-point digital hardware accelerator. The operation of the low-precision fixed-point digital hardware accelerator was assumed to be deterministic. The Brevitas⁵⁵ library was used to model the inference of a low-precision fixed-point digital hardware accelerator. Channel-wise quantization was performed where weight terms were quantized to a 4-bit fixed-point representation and activation terms were quantized to a 8-bit fixed-point representation.

PCM-based AIMC accelerator. For some experiments, the IBM HERMES Project Chip²⁹, a 64-core AIMC chip based on back-end integrated PCM in a 14-nm Complementary Metal-Oxide Semiconductor (CMOS) process with a total capacity of 4, 194, 304 programmable weights, was used. For others, when using this hardware was not possible, a representative hardware model was used instead. The operation of this model was verified by matching experimental data from distinct noise sources and high-level behaviors from hardware experiments. ADC and DAC circuits were assumed to operate ideally at 8-bit precision. Weight (programming) noise was modeled by fitting a third-order polynomial function of the mean error (standard

deviation) with respect to normalized weight values. For each normalized weight value, the error was assumed to be Gaussian distributed. Read noise was modeled using a lookup table, where the standard deviation (also assumed to be Gaussian distributed) of the read noise was recorded as a function of the conductance (in ADC units). Finally, output noise was modeled by adding a Gaussian distributed additive noise, to each column for each mini-batch. Other non-linear residual noise sources were assumed to be negligible.

Modified PCM-based AIMC accelerator. We performed experiments to determine the dominant source of adversarial robustness. The aforementioned AIMC chip model was modified to consider only a single stochastic noise source. To set the noise magnitude to produce a desired average test set accuracy value, Bayesian optimization was performed using Optuna⁵⁶. Specifically, 100 trials were executed with the Tree-structured Parzen Estimator (TPESampler) and the Median pruning algorithm (MedianPruner) optimization algorithms.

Adversarial success rate evaluation

As mentioned, to evaluate the ASR, we rely on the ASR metric. In terms of accuracy, the ASR can be determined using Algorithm 1.

Algorithm 1. Determination of the Adversarial Success Rate (ASR) metric.

Require: The model, `model`, clean inputs, `x`, clean targets, `y`, adversarial inputs, `xadv`

Ensure: The Adversarial Success Rate (ASR)

```
total_adversarial, correct_adversarial = 0
for (x,y,xadv) in zip(x,y,xadv) do    ▷ Iterate over each batch of
                                     the clean inputs, clean tar-
                                     gets, and adversarial inputs
    predicted_clean = model(x), predicted_adversarial =
    model(xadv)    ▷ Compute clean and adversarial predictions
    predicted_adversarial = predicted_adversarial
    [predicted_clean == y]    ▷ Mask adversarial predictions
    target_adversarial = target[predicted_clean == y]
    ▷ Mask adversarial targets
    total_adversarial += len(target_adversarial)
    correct_adversarial += sum(predicted_adversarial !=
    target_adversarial)    ▷ Increment the number of masked,
                                     adversarial predictions that are not
                                     matched with the desired, i.e., original,
                                     targets
end for
return correct_adversarial / total_adversarial
```

Attack generation and evaluation

To generate the PGD, Square, and OnePixel attacks, the Adversarial Robustness Toolbox (ART) was used. To generate the GBDA attacks, the following code repository was used: <https://github.com/facebookresearch/text-adversarial-attack>. All validation samples were processed sequentially, i.e., to generate adversarial inputs, a unit batch size was used. For each attack type, two attack parameters were varied –directly relating to the attack magnitude and number of iterations. To determine an appropriate range for each attack parameter, first, a large range for each parameter was explored. The upper bound was determined for each parameter by manually inspecting and labeling adversarial inputs. Then, the parameter space for which the ground truth labels differed was avoided. Finally, the lower bound was determined for each parameter by manually inspecting contour plots of the ASR and determining where the rate of change of the ASR plateaued.

For all experiments, when adversarial inputs were presented, they were interleaved with their corresponding standard inputs. This was done for two reasons. First, to determine the ASR, and second, to counteract temporal effects, such as drift. All other attack parameters

were set to default values as of commit ID de99dca9e0482b43-d2e5118f76d1b07135fcdad51 (<https://github.com/Trusted-AI/adversarial-ai-robustness-toolbox>).

For all target platforms, we consider the strongest scenario (i.e., the generation and evaluation of adversarial inputs using the same platform). Hence, all experiments consider four sets of network parameters, as for all target platforms, it is not possible to represent the weights in the exact same representation, i.e., for the AIMC chip and corresponding model, weights are encoded as conductance values, and for the original model they are represented using floating-point values: (i) the original floating-point weights, (ii) floating-point precision HWA retrained weights, (iii) HWA retrained, PTQ weights, and (iv) HWA retrained weights mapped to conductance values. For the AIMC chip, the mapped weights of the HWA-trained network were not used directly, as to mitigate mismatches in the assumed hardware configuration (during HWA retraining) and the AIMC chip, a number of post-mapping calibration steps are performed. We report the test set accuracy for all configurations (sets) of network parameters and platforms in Supplementary Table I.

Model deployment and inference for the AIMC chip

Model deployment and inference for the AIMC chip were performed using a sophisticated software stack, which allows for end-to-end deployment of Deep Learning (DL) models. First, using the torch.fx⁵⁷ library, PyTorch models are traced using symbolic tracing, which propagates proxy objects through them. Graph-based representations of models are formed, where vertices represent operations and directed edges represent connections between sequences of operations.

Second, pipelining is performed at the operation level of abstraction. Operations are grouped into distinct chains and branches. Chains are comprised of one or more branches, which execute in parallel. Operations within branches execute sequentially. The inputs of the branches are either the output values of the previous chain or cached values stored previously during the execution. At the first pipeline stage, new input is fed to the first chain and executed. At the second pipeline stage, new input is once again fed to the first chain and executed, however, the output(s) of the first chain is(are) fed to the second chain and executed. This process continues until the pipeline is full, and for each pipeline stage, all chains are executed. After the last input has been fed to the first chain, the pipeline is flushed until the final output has been received. This process maximizes the number of parallel MVMs performed on-chip.

Next, the floating-point weights of mapable layers (to AIMC tiles) from the input DL model are mapped to programmable conductance states, ranging from 0 to $G_{\max}$, where $G_{\max}$ is typically set to 160 μS . Post-training optimizations⁵⁸ are then performed to tune the (i) input range of each AIMC tile and the (ii) maximum conductance range of each column. After these optimizations are performed, before the tuned target conductance states are programmed to the devices using Gradient Descent Programming (GDP)^{41,59}, they are stored in floating-point precision to be deployed on other hardware platforms. Finally, the software stack handles the execution of the compiled model by interfacing itself with lower-level drivers, which handle the execution of the operations on the AIMC chip.

Hardware-in-the-loop adversarial attacks targeting the AIMC chip.

To perform hardware-in-the-loop attacks targeting the AIMC chip, for the white-box PGD attack, (i) the inputs and (ii) the weights of each layer, in addition to (iii) the outputs of the network, are required to compute gradients. For inherently stochastic systems, this is problematic, as the programmed weights differ from the desired weights. Moreover, weight values randomly fluctuate during inference operation due to high- and low-frequency weight noise, and conductance drift. As described in Eq. (1), representative weights (associated with a given period of elapsed time) must be inferred by solving an

optimization problem. For the semi-black-box OnePixel attack, probability labels and output logits are required as well.

For both of these types of attacks, before the attack was performed, HWA weights were programmed to conductance values and post-training optimizations were performed, as described in Model Deployment and Inference for the AIMC chip. For both of these types of attacks, adversarial inputs were generated by iteratively inferring inputs using the AIMC chip, performing backwards propagation, and using the resulting information to modify the adversarial inputs. Hence, the generation of one input in a sequence is dependent on the output of the previous input in a sequence. This is also problematic, as it means that these attacks cannot be pipelined. Consequently, they take a relatively long time to run, and are susceptible to conductance drift. As a consequence, all hardware-in-the-loop adversarial attacks targeting the AIMC chip are generated and evaluated for a randomly sampled subset (1000/10,000) of the test set, where the number of samples in each class is equal (100). The same split was used for all associated experiments.

To perform both types of attacks, the following steps were performed. First, initial inputs, as determined by the attack algorithm, and clean inputs, were constructed. These were propagated through the AIMC chip. During inference, for white-box attacks, the inputs to each layer were cached. For the AIMC chip, inputs and outputs of each layer can easily be cached using our software stack. In a practical setting, voltage traces could be probed, or a side-channel attack could be performed to obtain these values. Representative weight values were inferred by solving the optimization problem described in Eq. (1) by performing 2000 MVMs on each crossbar, with inputs sampled from a clipped Gaussian distribution. Again, in a practical setting, even if the attacker does not have direct control of the inputs and outputs of each crossbar, these could be measured during normal operation and Eq. (1) could be applied. For both types of attacks, the output logits were also cached.

Next, the PyTorch⁶⁰ library was used to construct a proxy graph of the original network deployed on the AIMC chip. Random inputs were propagated through this proxy graph to construct a computational graph used for back-propagation. The cached inputs and weights to each layer were overridden using the cached values. The output logits were also overridden. Gradients were computed using the auto-grad functionality of PyTorch on the proxy graph, and the gradients, in addition to the output logits, were fed to the attack algorithm to modify the current adversarial inputs to generate the next batch of inputs to infer. This process was repeated for each attack iteration, and for each batch of inputs.

Data availability

The raw data that support the findings of this study can be made available by the corresponding authors upon request after IBM management approval.

Code availability

The code used to perform the simulations included in this study can be made available by the corresponding authors upon request after IBM management approval.

References

1. Akhtar, N. & Mian, A. Threat of adversarial attacks on deep learning in computer vision: a survey. *IEEE Access* **6**, 14410–14430 (2018).
2. Pitropakis, N., Panaousis, E., Giannetsos, T., Anastasiadis, E. & Loukas, G. A taxonomy and survey of attacks against machine learning. *Comput. Sci. Rev.* **34**, 100199 (2019).
3. Ghaffari Laleh, N. et al. Adversarial attacks and adversarial robustness in computational pathology. *Nat. Commun.* **13**, 5711 (2022).
4. Woods, W., Chen, J. & Teuscher, C. Adversarial explanations for understanding image classification decisions and improved neural network robustness. *Nat. Mach. Intell.* **1**, 508–516 (2019).

5. Pereira, G. T. & de Carvalho, A. C. P. L. F. Bringing robustness against adversarial attacks. *Nat. Mach. Intell.* **1**, 499–500 (2019).
6. Ielmini, D. & Wong, H.-S. P. In-memory computing with resistive switching devices. *Nat. Electron.* **1**, 333–343 (2018).
7. Sebastian, A., Gallo, M. L., Khaddam-Aljameh, R. & Eleftheriou, E. Memory devices and applications for in-memory computing. *Nat. Nanotechnol.* **15**, 529–544 (2020).
8. Lanza, M. et al. Memristive technologies for data storage, computation, encryption, and radio-frequency communication. *Science* **376**, eabj9979 (2022).
9. Zidan, M. A., Strachan, J. P. & Lu, W. D. The future of electronics based on memristive systems. *Nat. Electron.* **1**, 22–29 (2018).
10. Mehonic, A. et al. Memristors—from in-memory computing, deep learning acceleration, and spiking neural networks to the future of neuromorphic and bio-inspired computing. *Adv. Intell. Syst.* **2**, 2000085 (2020).
11. Joshi, V. et al. Accurate deep neural network inference using computational phase-change memory. *Nat. Commun.* **11**, 2473 (2020).
12. Rasch, M. J. et al. Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators. *Nat. Commun.* **14**, 5282 (2023).
13. Bhattacharjee, A. & Panda, P. Rethinking non-idealities in memristive crossbars for adversarial robustness in neural networks. Preprint at CoRR <https://arxiv.org/abs/2008.11298> (2020).
14. Roy, D., Chakraborty, I., Ibrayev, T. & Roy, K. On the intrinsic robustness of NVM crossbars against adversarial attacks. In *ACM/IEEE Design Automation Conference (DAC)* 565–570 <https://doi.org/10.1109/dac18074.2021.9586202> (San Diego, CA, 2021).
15. Tao, C., Roy, D., Chakraborty, I. & Roy, K. On noise stability and robustness of adversarially trained networks on NVM crossbars. In *IEEE Transactions on Very Large Scale Integration* **30** 1448–1460 (IEEE, 2022).
16. Cherupally, S. K. et al. Improving the accuracy and robustness of RRAM-based in-memory computing against RRAM hardware noise and adversarial attacks. *Semicond. Sci. Technol.* **37**, 034001 (2022).
17. Paudel, B. R. & Tragoudas, S. The impact of on-chip training to adversarial attacks in memristive crossbar arrays. In *IEEE International Test Conference (ITC)* 519–523 <https://doi.org/10.1109/itc50671.2022.00064> (Anaheim, CA, 2022).
18. Shang, L., Jung, S., Li, F. & Pan, C. Fault-aware adversary attack analyses and enhancement for RRAM-based neuromorphic accelerator. *Front. Sens.* **3**, 896299 (2022).
19. McLemore, T. et al. Exploiting device-level non-idealities for adversarial attacks on ReRAM-based neural networks. *Mem. Mater. Devices Circuits Syst.* **4**, 100053 (2023).
20. Lv, H., Li, B., Wang, Y., Liu, C. & Zhang, L. VADER: leveraging the natural variation of hardware to enhance adversarial attack. In *Asia and South Pacific Design Automation Conference (ASP-DAC)* 487–492 <https://doi.org/10.1145/3394885.3431598> (Tokyo, Japan, 2021).
21. Lv, H., Li, B., Zhang, L., Liu, C. & Wang, Y. Variation enhanced attacks against RRAM-based neuromorphic computing system. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **42**, 1588–1596 (2023).
22. Liu, Y. et al. RoBERTa: a robustly optimized BERT pretraining approach. Preprint at CoRR <http://arxiv.org/abs/1907.11692> (2019).
23. Williams, A., Nangia, N. & Bowman, S. A Broad-coverage challenge corpus for sentence understanding through inference. In *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* 1112 <https://doi.org/10.18653/v1/n18-1101> (New Orleans, LA, 2018).
24. Chakraborty, A., Alam, M., Dey, V., Chattopadhyay, A. & Mukhopadhyay, D. A survey on adversarial attacks and defences. *CAAI Trans. Intell. Technol.* **6**, 25–45 (2021).
25. Biggio, B., Nelson, B. & Laskov, P. Poisoning attacks against support vector machines. In *International Conference on Machine Learning (ICML)* 1467–1474 <https://doi.org/10.5555/3042573.3042761> (Madison, WI, 2012).
26. Wenger, E. et al. Backdoor attacks against deep learning systems in the physical world. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 6206–6215 <https://doi.org/10.1109/cvpr46437.2021.00614> (Nashville, TN, 2021).
27. Standaert, F.-X. Introduction to side-channel attacks. In *Secure Integrated Circuits and Systems* 27–42 https://doi.org/10.1007/978-0-387-71829-3_2 (Boston, MA, 2010).
28. Wang, Z., Meng, F.-H., Park, Y., Eshraghian, J. K. & Lu, W. D. Side-channel attack analysis on in-memory computing architectures. *IEEE Trans. Emerg. Top. Comput.* **12**, 109–121 (2024).
29. Le Gallo, M. et al. A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference. *Nat. Electron.* **6**, 680–693 (2023).
30. Nardone, M., Kozub, V., Karpov, I. & Karpov, V. Possible mechanisms for 1/f noise in chalcogenide glasses: a theoretical description. *Phys. Rev. B* **79**, 165206 (2009).
31. Madry, A., Makelov, A., Schmidt, L., Tsipras, D. & Vladu, A. Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations (ICLR)* <https://doi.org/10.48550/arxiv.1706.06083> (New Orleans, LA, 2019).
32. Andriushchenko, M., Croce, F., Flammarion, N. & Hein, M. Square attack: a query-efficient black-box adversarial attack via random search. In *European Conference on Computer Vision (ECCV)* 484–501 https://doi.org/10.1007/978-3-030-58592-1_29 (Glasgow, Scotland, 2020).
33. Su, J., Vargas, D. V. & Sakurai, K. One pixel attack for fooling deep neural networks. *IEEE Trans. Evolut. Comput.* **23**, 828–841 (2019).
34. Huang, T., Menkovski, V., Pei, Y. & Pechenizkiy, M. Bridging the performance gap between FGSM and PGD adversarial training. Preprint at CoRR <https://arxiv.org/abs/2011.05157> (2020).
35. Tramer, F. et al. Ensemble adversarial training: attacks and defenses. In *International Conference on Learning Representations (ICLR)* <https://doi.org/10.48550/arxiv.1705.07204> (Vancouver, Canada, 2020).
36. Carlini, N. et al. On evaluating adversarial robustness. Preprint at CoRR <http://arxiv.org/abs/1902.06705> (2019).
37. Drolet, P. et al. Hardware-aware training techniques for improving robustness of ex-situ neural network transfer onto passive TiO₂ ReRAM Crossbars. Preprint at CoRR (2023).
38. Stutz, D., Chandramoorthy, N., Hein, M. & Schiele, B. Random and adversarial bit error robustness: energy-efficient and secure DNN accelerators. *IEEE Trans. Pattern Anal. Mach. Intell.* **45**, 3632–3647 (2023).
39. Shumailov, I., Zhao, Y., Mullins, R. & Anderson, R. To compress or not to compress: understanding the interactions between adversarial attacks and neural network compression. In *Machine Learning and Systems (MLSys)* 230–240 <https://doi.org/10.48550/arxiv.1810.00208> (Stanford, CA, 2019).
40. Chakraborty, I., Fayed Ali, M., Eun Kim, D., Ankit, A. & Roy, K. GENIEx: A generalized approach to emulating non-ideality in memristive xbars using neural networks. In *ACM/IEEE Design Automation Conference (DAC)* 1–6 <https://doi.org/10.1109/dac18072.2020.9218688> (San Francisco, CA, 2020).
41. Büchel, J. et al. Programming weights to analog in-memory computing cores by direct minimization of the matrix-vector multiplication error. *IEEE J. Emerg. Sel. Top. Circuits Syst.* **13**, 1052–1061 (2023).
42. Yang, K., Joshua Yang, J., Huang, R. & Yang, Y. Nonlinearity in memristors for neuromorphic dynamic systems. *Small Sci.* **2**, 2100049 (2022).

43. Zhang, W. E., Sheng, Q. Z., Alhazmi, A. & Li, C. Adversarial attacks on deep-learning models in natural language processing: a survey. *ACM Trans. Intell. Syst. Technol.* **11**, 1–41 (2020).
44. Guo, C., Sablayrolles, A., J'egou, H. & Kiela, D. Gradient-based adversarial attacks against text transformers. In *Conference on Empirical Methods in Natural Language Processing* 5747–5757 <https://doi.org/10.18653/v1/2021.emnlp-main.464> (Punta Cana, Dominican Republic, 2021).
45. Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q. & Artzi, Y. BERT-Score: evaluating text generation with BERT. In *International Conference on Learning Representations (ICLR)* <https://doi.org/10.48550/arxiv.1904.09675> (Addis Ababa, Ethiopia, 2020).
46. Wang, Y. et al. Adversarial attacks and defenses in machine learning-empowered communication systems and networks: a contemporary survey. *IEEE Commun. Surv. Tutor.* **25**, 2245–2298 (2023).
47. Bai, T., Luo, J., Zhao, J., Wen, B. & Wang, Q. Recent advances in adversarial training for adversarial robustness. In *International Joint Conference on Artificial Intelligence (IJCAI)* 4312–4321 <https://doi.org/10.24963/ijcai.2021/591> (Montreal, Canada, 2021).
48. Ren, K., Zheng, T., Qin, Z. & Liu, X. Adversarial attacks and defenses in deep learning. *Engineering* **6**, 346–360 (2020).
49. Langenegger, J. et al. In-memory factorization of holographic perceptual representations. *Nat. Nanotechnol.* **18**, 479–485 (2023).
50. Harabi, K.-E. et al. A memristor-based Bayesian machine. *Nat. Electron.* **6**, 52–63 (2023).
51. Cai, F. et al. Power-efficient combinatorial optimization using intrinsic noise in memristor Hopfield neural networks. *Nat. Electron.* **3**, 409–418 (2020).
52. Krestinskaya, O., Bakambekova, A. & James, A. P. AMSNet: analog memristive system architecture for mean-pooling with dropout convolutional neural network. In *IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS)* 272–273 <https://doi.org/10.1109/aicas.2019.8771611> (Abu Dhabi, UAE, 2019).
53. Krestinskaya, O. & James, A. P. Analogue neuro-memristive convolutional dropout nets. *Proc. R. Soc. A Math. Phys. Eng. Sci.* **476**, 20200210 (2020).
54. Le Gallo, M. et al. Using the IBM analog in-memory hardware acceleration kit for neural network training and inference. *APL Mach. Learn.* **1**, 041102 (2023).
55. Pappalardo, A. Xilinx/Brevitas. Zenodo <https://doi.org/10.5281/zenodo.3333552> (2021).
56. Akiba, T., Sano, S., Yanase, T., Ohta, T. & Koyama, M. Optuna: a next-generation hyperparameter optimization framework. In *ACM International Conference on Knowledge Discovery & Data Mining (SIGKDD)* 2623–2631 <https://doi.org/10.1145/3292500.3330701> (Anchorage, AK, 2019).
57. Reed, J. K., DeVito, Z., He, H., Ussery, A. & Ansel, J. torch.fx: practical program capture and transformation for deep learning in Python. In *Proceedings of Machine Learning and Systems*. (Eds Marculescu, D., Chi, Y. & Wu, C.) Vol 4, 638–651 (2022).
58. Lammie, C. et al. Improving the accuracy of analog-based in-memory computing accelerators post-training. In *2024 IEEE International Symposium on Circuits and Systems (ISCAS)* 1–5 <https://doi.org/10.1109/iscas58744.2024.10558540> (Singapore, Singapore, 2024).
59. Vasilopoulos, A. et al. Exploiting the state dependency of conductance variations in memristive devices for accurate in-memory computing. *IEEE Trans. Electron Devices* **70**, 6279–6285 (2023).
60. Paszke, A. et al. PyTorch: an imperative style, high-performance deep learning library. In *Neural Information Processing Systems (NeurIPS)* 8024–8035 <https://doi.org/10.48550/arxiv.1912.01703> (Vancouver, Canada, 2019).

Acknowledgements

This work was supported by the IBM Research AI Hardware Center. We would like to thank Benedikt Kersting, Irem Boybat, Hadjer Benmeziane, and Giacomo Camposampiero, for fruitful discussions. We would also like to thank Vijay Narayanan and Jeff Burns for managerial support.

Author contributions

C.L. and A.S. initiated the project. C.L. designed and planned the project. C.L. and J.B. set up the infrastructure for generating and evaluating the adversarial attacks. C.L., J.B., and A.V. set up the infrastructure for automatically deploying trained models on the IBM Hermes Project Chip. C.L. wrote the manuscript with input from all authors. M.L.G. and A.S. supervised the project.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s41467-025-56595-2>.

Correspondence and requests for materials should be addressed to Corey Lammie or Abu Sebastian.

Peer review information *Nature Communications* thanks Wei Ni, Alex James, and the other, anonymous, reviewer(s) for their contribution to the peer review of this work. A peer review file is available.

Reprints and permissions information is available at <http://www.nature.com/reprints>

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025